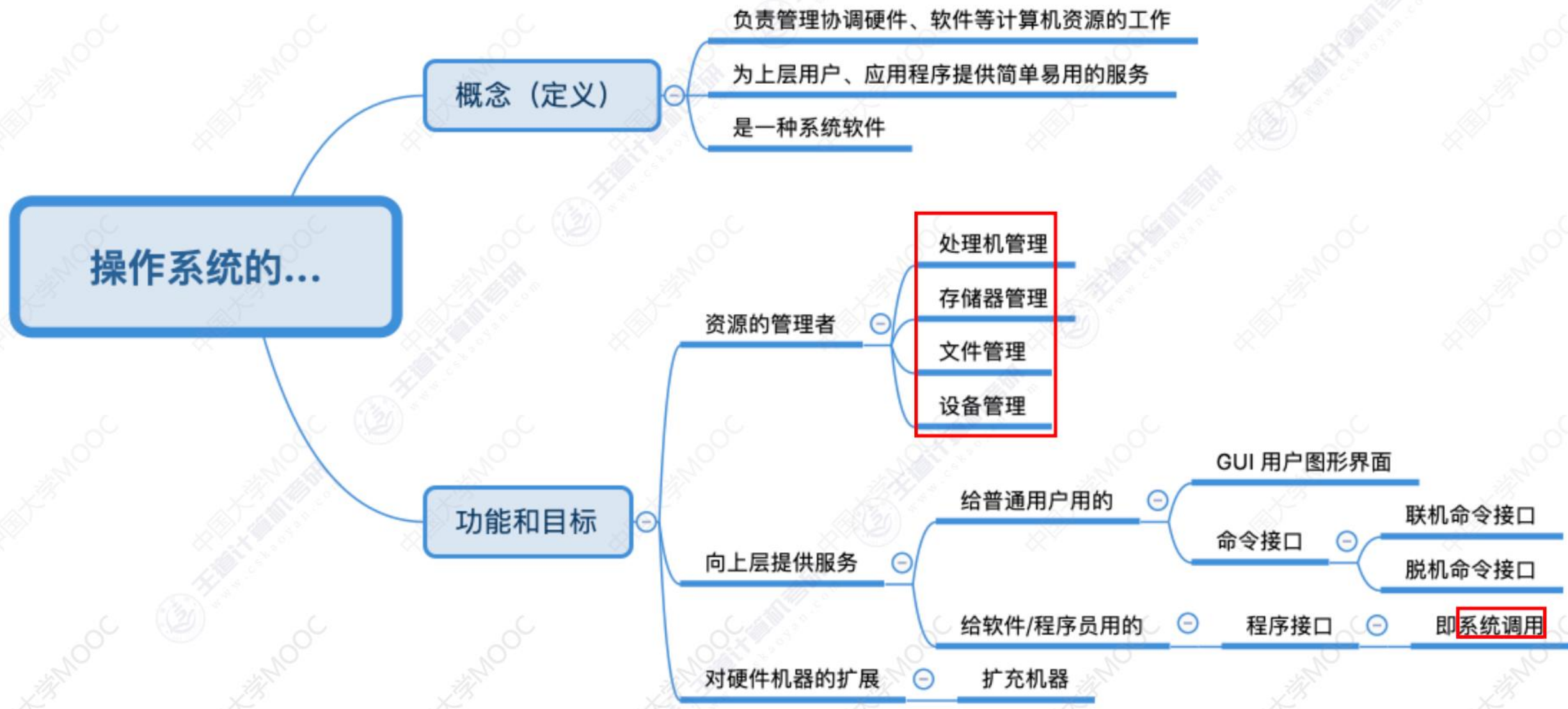


知识回顾与重要考点



知识回顾与重要考点



重要考点:

理解并发和并行的区别

并发和共享互为存在条件

没有并发和共享,就谈不上虚拟和异步,因此并发和共享是操作系统的两个最基本的特征

知识回顾与重要考点

OS的发展与分类

手工操作阶段

缺：人机速度矛盾

批处理阶段

单道批处理系统（引入脱机输入输出技术）

优：缓解人机速度矛盾

缺：资源利用率依然很低

多道批处理系统（操作系统开始出现）

优：多道程序并发执行，资源利用率高

缺：不提供人机交互功能

分时操作系统

优：提供人机交互功能

缺：不能优先处理紧急任务

实时操作系统

硬实时系统

必须在绝对严格的规定时间内完成处理

软实时系统

能接受偶尔违反时间规定

优：能优先处理紧急任务

网络操作系统

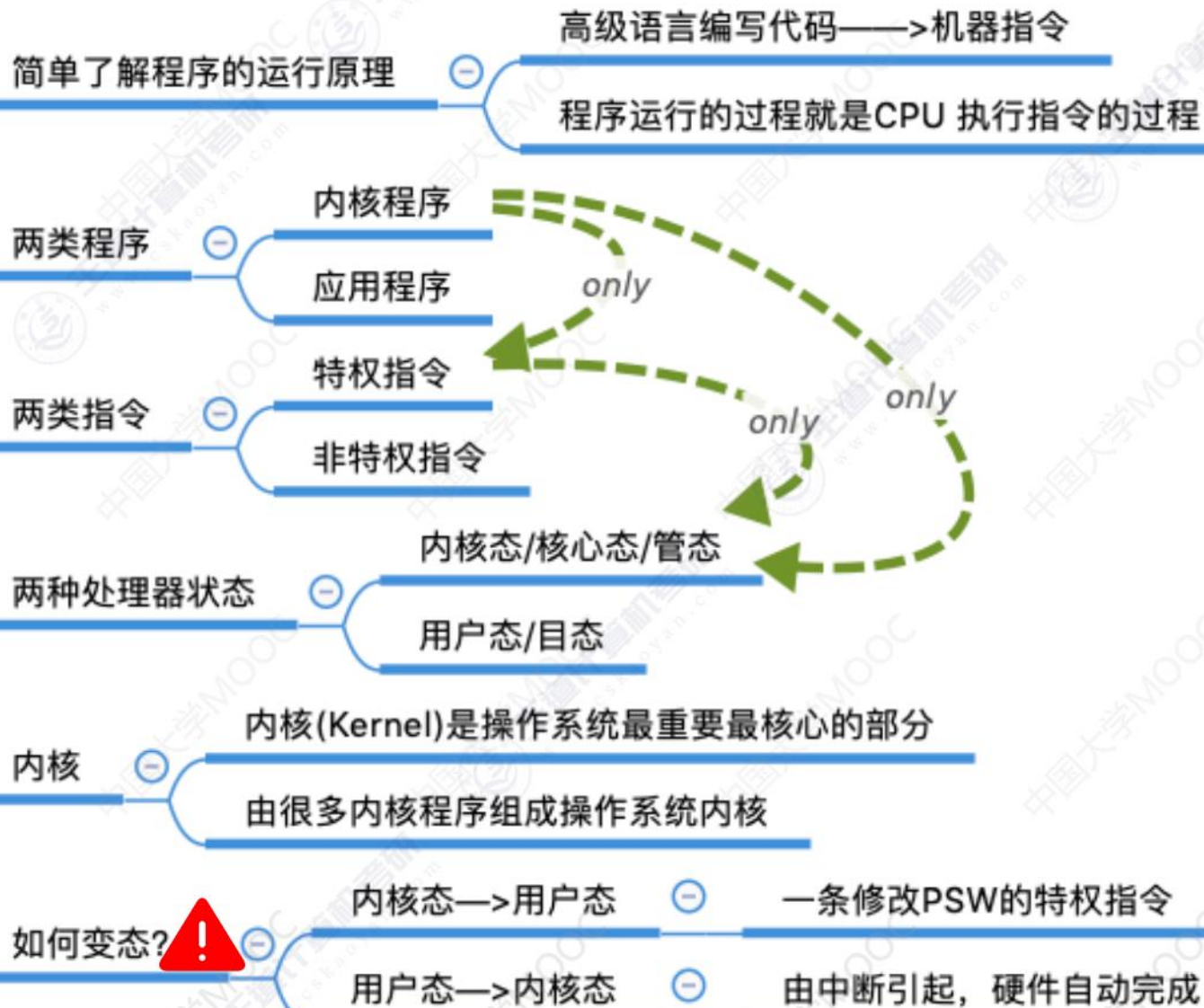
分布式操作系统

个人计算机操作系统

知识回顾与重要考点



操作系统的运行机制

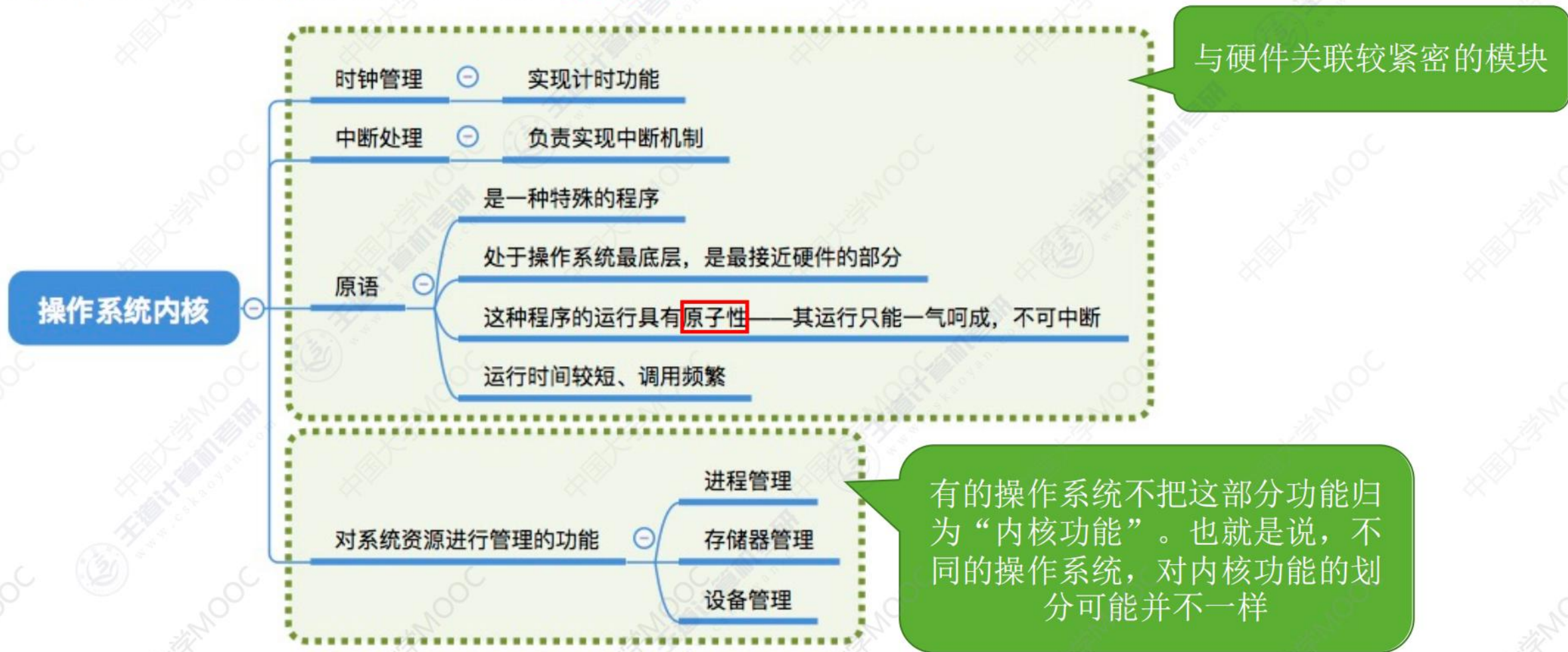


Tips:

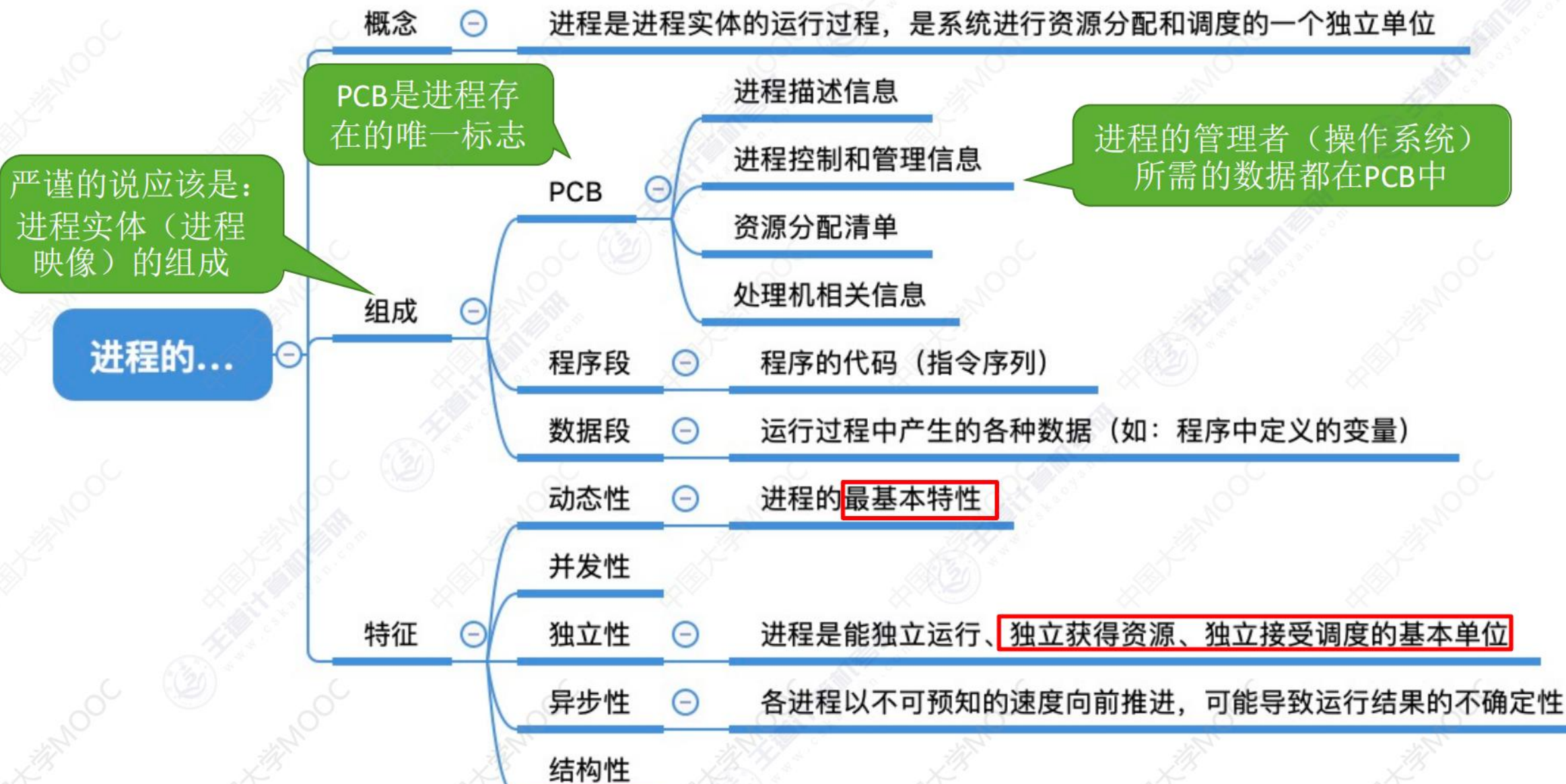
1. 都是高频考点, 很重要
2. 初学者不完全理解没关系, 放心大胆地往后学, 随着后面章节的学习, 理解会逐渐加深

操作系统的内核

内核是计算机上配置的底层软件，是操作系统最基本、最核心的部分。实现操作系统内核功能的那些程序就是内核程序。



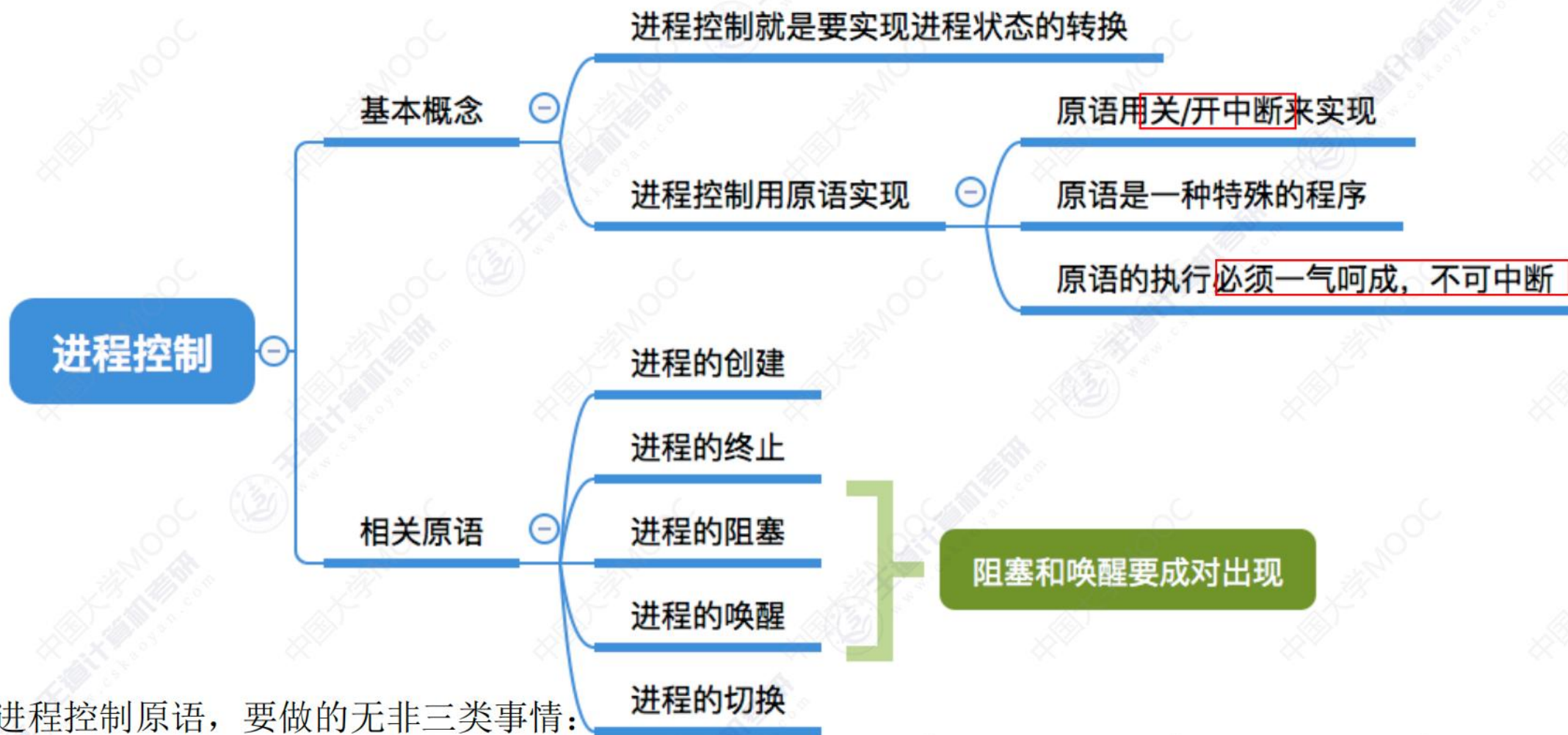
知识回顾与重要考点



知识回顾与重要考点



知识回顾与重要考点



无论哪个进程控制原语, 要做的无非三类事情:

1. 更新PCB中的信息
2. 将PCB插入合适的队列
3. 分配/回收资源

修改进程状态 (state)
保存/恢复运行环境

进程控制相关的原语

操作系统创建一个进程时使用的原语

进程的创建

创建原语

申请空白PCB

为新进程分配所需资源

初始化PCB

将PCB插入就绪队列

创建态→就绪态

引起进程创建的事件

用户登录

分时系统中，用户登录成功，系统会建立为其建立一个新的进程

作业调度

多道批处理系统中，有新的作业放入内存时，会为其建立一个新的进程

提供服务

用户向操作系统提出某些请求时，会新建一个进程处理该请求

应用请求

由用户进程主动请求创建一个子进程

进程控制相关的原语

就绪态/阻塞态/运行态
→终止态→无

撤销原语

从PCB集合中找到终止进程的PCB

若进程正在运行，立即剥夺CPU，将CPU分配给其他进程

终止其所有子进程

进程间的关系是树形结构

将该进程拥有的所有资源归还给父进程或操作系统

删除PCB

进程的终止

引起进程终止的事件

正常结束

进程自己请求终止（exit系统调用）

异常结束

整数除以0、非法使用特权指令，
然后被操作系统强行杀掉

外界干预

Ctrl+Alt+delete，用户选择杀掉进程

进程控制相关的原语

阻塞原语唤醒原语必须成对使用

进程的阻塞和唤醒

运行态→
阻塞态

进程的阻塞

阻塞原语

找到要阻塞的进程对应的PCB

保护进程运行现场，将PCB状态信息设置为“阻塞态”，暂时停止进程运行

将PCB插入相应事件的等待队列

引起进程阻塞的事件

需要等待系统分配某种资源

需要等待相互合作的其他进程完成工作

阻塞态→
就绪态

进程的唤醒

唤醒原语

在事件等待队列中找到PCB

将PCB从等待队列移除，设置进程为就绪态

将PCB插入就绪队列，等待被调度

引起进程唤醒的事件

等待的事件发生

因何事阻塞，就应由何事唤醒

进程控制相关的原语

进程上下文 (Context)

运行态→就绪态
就绪态→运行态

进程的切换

切换原语

将运行环境信息存入PCB

PCB移入相应队列

选择另一个进程执行，并更新其PCB

根据PCB恢复新进程所需的运行环境

引起进程切换的事件

当前进程时间片到

有更高优先级的进程到达

当前进程主动阻塞

当前进程终止

知识回顾与重要考点

进程通信

共享存储

设置一个共享内存区域，并映射到进程的虚拟地址空间

要互斥地访问共享空间（由通信进程自己负责实现互斥）

两种方式

基于数据结构（低级）

基于存储区的共享（高级）

消息传递

传递结构化的消息（消息头/消息体）

系统提供“发送/接受原语”

两种方式

直接通信方式

消息直接挂到接收进程的消息队列里

间接（信箱）通信方式

消息先发 to 中间体（信箱）

管道通信

设置一个特殊的共享文件（管道），其实就是一个内存缓冲区

一个管道只能实现半双工通信

实现双向同时通信要建立两个管道

各进程要互斥访问管道（由操作系统负责实现互斥）

管道写满时，写进程阻塞。管道读空时，读进程阻塞

知识总览

线程、多线程模型

什么是线程，为什么要引入线程？

引入线程机制后，有什么变化？

线程有哪些重要的属性

线程的实现方式

用户级线程

内核级线程

多线程模型

多对一模型

一对一模型

多对多模型

什么是线程，为什么要引入线程？



有的进程可能需要“同时”做很多事，而传统的进程只能串行地执行一系列程序。为此，引入了“线程”，来增加并发度。

代码1
代码2
代码3
.....

进程1

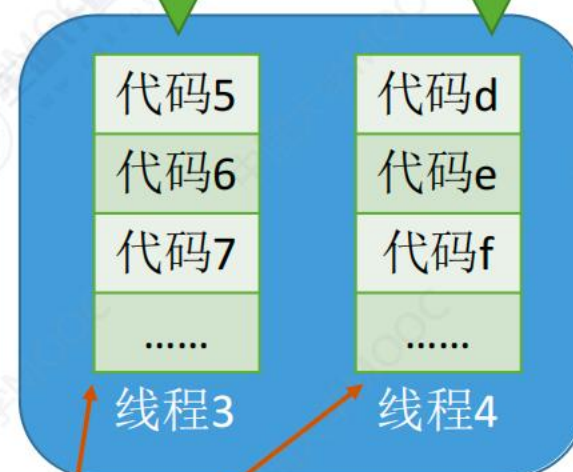
代码a
代码b
代码c
.....

进程2

传统的进程是程序执行流的最小单位



进程

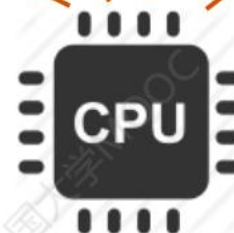


进程

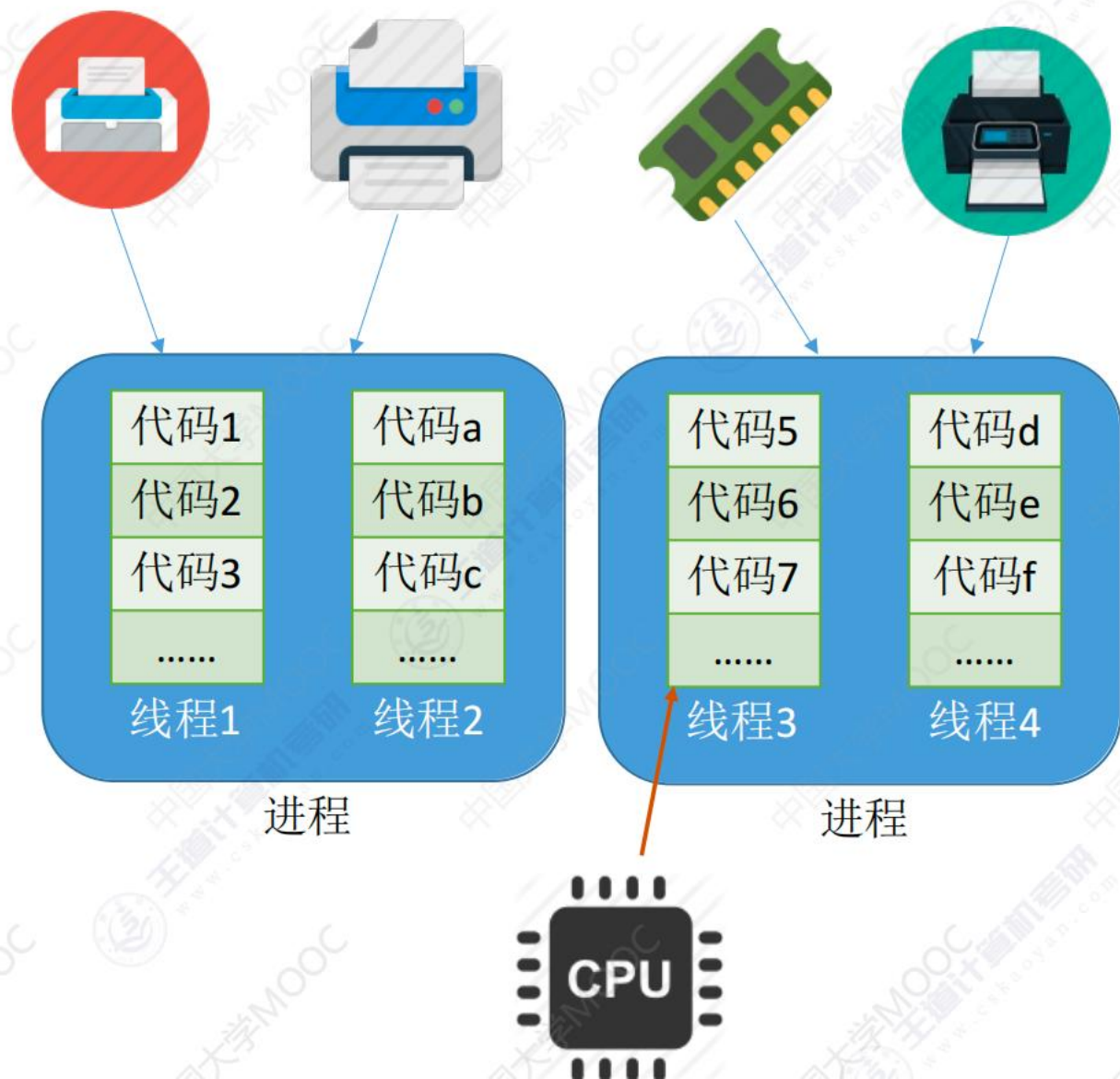
QQ视频聊天处理程序

QQ传送文件处理程序

引入线程后，线程成为了程序执行流的最小单位



什么是线程，为什么要引入线程？



可以把线程理解为“轻量级进程”。

线程是一个基本的CPU执行单元，也是程序执行流的最小单位。

引入线程之后，不仅是进程之间可以并发，进程内的各线程之间也可以并发，从而进一步提升了系统的并发度，使得一个进程内也可以并发处理各种任务（如QQ视频、文字聊天、传文件）

引入线程后，进程只作为除CPU之外的系统资源的分配单元（如打印机、内存地址空间等都是分配给进程的）。

线程则作为处理机的分配单元。

引入线程机制后，有什么变化？

带来的变化

资源分配、调度

传统进程机制中，进程是资源分配、调度的基本单位

引入线程后，进程是资源分配的基本单位，线程是调度的基本单位

并发性

传统进程机制中，只能进程间并发

引入线程后，各线程间也能并发，提升了并发度

系统开销

传统的进程间并发，需要切换进程的运行环境，系统开销很大

线程间并发，如果是同一进程内的线程切换，则不需要切换进程环境，系统开销小

引入线程后，并发所带来的系统开销减小

类比：

去图书馆看书。桌子=处理机，人=进程，看不同的书=线程
切换进程运行环境：有一个不认识的人要用桌子，你需要把你的书收走，他把自己的书放到桌上

同一进程内的线程切换=你的舍友要用这张书桌，可以不把桌子上的书收走



知识回顾与重要考点

处理机调度

基本概念

按某种算法选择一个进程将处理机分配给它

三个层次

高级调度 (作业调度)

按照某种规则 从后备队列中选择合适的作业将其调入内存，并为其创建进程

中级调度 (内存调度)

按照某种规则，从挂起队列中选择合适的进程将其数据调回内存

低级调度 (进程调度)

按照某种规则 从就绪队列中选择一个进程为其分配处理机

三层调度的联系、对比

高级调度

外存—>内存 (面向作业)

发生频率：最低

中级调度

外存—>内存 (面向进程)

发生频率：中等

低级调度

内存—>CPU

发生频率：最高

调度算法要研究的问题

补充知识

为减轻系统负载，提高资源利用率，暂时不执行的进程会被调到外存从而变为“挂起态”

七状态模型：在五状态模型的基础上加入了“就绪挂起”和“阻塞挂起”两种状态

知识回顾与重要考点

调度算法的评价指标

要理解，并且会计算

CPU利用率

也有题目会让我们计算某设备的利用率

$$\text{利用率} = \frac{\text{忙碌的时间}}{\text{总时间}}$$

系统吞吐量

$$\text{系统吞吐量} = \frac{\text{总共完成了多少道作业}}{\text{总共花了多少时间}}$$

周转时间

$$\text{周转时间} = \text{作业完成时间} - \text{作业提交时间}$$

$$\text{平均周转时间} = \frac{\text{各作业周转时间之和}}{\text{作业数}}$$

$$\text{带权周转时间} = \frac{\text{作业周转时间}}{\text{作业实际运行的时间}}$$

$$\text{平均带权周转时间} = \frac{\text{各作业带权周转时间之和}}{\text{作业数}}$$

等待时间

进程/作业 等待被服务的时间之和

平均等待时间即各个 进程/作业 等待时间的平均值

响应时间

从用户提交请求到首次产生响应所用的时间



知识回顾与重要考点

进程调度的...

时机

什么时候需要进程调度?

主动放弃

进程正常终止

运行过程中发生异常而终止

主动阻塞 (如 等待I/O)

被动放弃

分给进程的时间片用完

有更紧急的事情需要处理 (如 I/O中断)

有更高优先级的进程进入就绪队列

什么时候不能进行进程调度?

在处理中断的过程中

进程在操作系统内核程序临界区中

原子操作过程中 (原语)

切换与过程

狭义的“调度”和“切换”的区别

切换过程

对原来运行进程各种数据的保存

对新的进程各种数据的恢复

重要结论: 进程调度、切换是有代价的 并不是调度越频繁, 并发度就越高

方式

非剥夺调度方式 (非抢占式)

只能由当前运行的进程主动放弃CPU

剥夺调度方式 (抢占式)

可由操作系统剥夺当前进程的CPU使用权

闲逛进程

调度程序永远的备胎，没有其他就绪进程时，运行闲逛进程（idle）

idle 

英 ['aɪd(ə)] 

美 ['aɪd(ə)] 

adj. 无事可做的；懒惰的；闲置的；空闲的；琐碎无聊的，毫无意义的；虚张声势的，唬人的；（钱）现金保存的，存于无息账户的

v. 闲混，无所事事；使……停止工作，使闲散；（发动机、车辆）空转

闲逛进程的特性：

- 优先级最低
- 可以是0地址指令，占一个完整的指令周期（指令周期末尾例行检查中断）
- 能耗低

知识回顾与重要考点

算法	思想&规则	可抢占?	优点	缺点	考虑到等待时间&运行时间?	会导致饥饿?
FCFS	自己回忆	非抢占式	公平; 实现简单	对短作业不利	等待时间√ 运行时间×	不会
SJF/S PF	自己回忆	默认为非抢占式, 也有SJF的抢占式 版本最短剩余时间 优先算法 (SRTN)	“最短的” 平均等待 /周转时间;	对长作业不利, 可 能导致饥饿; 难以 做到真正的短作业 优先	等待时间×	会
HRRN	自己回忆	非抢占式	上述两种算法的权衡 折中, 综合考虑的等 待时间和运行时间		等待时间√ 运行时间√	不会

注: 这几种算法主要关心对用户的公平性、平均周转时间、平均等待时间等评价系统整体性能的指标, 但是不关心“响应时间”, 也并不区分任务的紧急程度, 因此对于用户来说, 交互性很糟糕。因此这三种算法一般适合用于**早期的批处理系统**, 当然, **FCFS**算法也常结合其他的算法使用, 在现在也扮演着很重要的角色。而适合用于**交互式系统**的调度算法将在下一个小节介绍...

知识回顾与重要考点

算法	思想&规则	可抢占?	优点	缺点	会导致饥饿?	补充
时间片轮转		抢占式	公平, 适用于分时系统	频繁切换有开销, 不区分优先级	不会	时间片太大或太小有何影响?
优先级调度		有抢占式的, 也有非抢占式的。注意做题时的区别	区分优先级, 适用于实时系统	可能导致饥饿	会	动态/静态优先级。各类型进程如何设置优先级? 如何调整优先级?
多级反馈队列	较复杂, 注意理解	抢占式	平衡优秀 666	一般不说它有缺点, 不过可能导致饥饿	会	

注: 比起早期的批处理操作系统来说, 由于计算机造价大幅降低, 因此之后出现的交互式操作系统 (包括分时操作系统、实时操作系统等) 更注重系统的响应时间、公平性、平衡性等指标。而这几种算法恰好也能较好地满足交互式系统的需求。因此这三种算法适合用于**交互式系统**。(比如UNIX使用的就是多级反馈队列调度算法)

知识回顾与重要考点

并发性带来了异步性，有时需要通过进程同步解决这种异步问题。
有的进程之间需要相互配合地完成工作，各进程的工作推进需要遵循一定的先后顺序。

进程同步

对临界资源的访问，需要互斥的进行。即同一时间段内只能允许一个进程访问该资源

同步、互斥

进程互斥

四个部分

进入区 检查是否可进入临界区，若可进入，需要“上锁”

临界区 访问临界资源的那段代码

退出区 负责“解锁”

剩余区 其余代码部分

需要遵循的原则

空闲让进 临界区空闲时，应允许一个进程访问

忙则等待 临界区正在被访问时，其他试图访问的进程需要等待

有限等待 要在有限时间内进入临界区，保证不会饥饿

让权等待 进不了临界区的进程，要释放处理机，防止忙等

知识回顾与重要考点

进程互斥的 软件实现方法

单标志法

在进入区只做“检查”，不“上锁”

在退出区把临界区的使用权转交给另一个进程
(相当于在退出区既给另一进程“解锁”，又给自己“上锁”)

主要问题：不遵循“空闲让进”原则

双标志先检查

在进入区先“检查”后“上锁”，退出区“解锁”

主要问题：不遵循“忙则等待”原则

双标志后检查

在进入区先“加锁”后“检查”，退出区“解锁”

主要问题：不遵循“空闲让进、有限等待”原则，可能导致“饥饿”

Peterson 算法

在进入区“主动争取—主动谦让—检查对方是否想进、己方是否谦让”

主要问题：不遵循“让权等待”原则，会发生“忙等”

知识回顾与重要考点

进程互斥的 硬件实现方法

中断屏蔽方法

使用“开/关中断”指令实现

优点：简单高效

缺点：只适用于单处理机；只适用于操作系统内核进程

TestAndSet (TS指令/TSL指令)

old 记录是否已被上锁；
再将 lock 设为 true；
检查临界区是否已被上锁
(若已上锁，则循环重复前几步)

优点：实现简单；适用于多处理机环境；

缺点：不满足“让权等待”

逻辑上同TSL

Swap指令 (XCHG指令)

知识回顾与重要考点

信号量机制

整型信号量

用一个整数型变量作为信号量，数值表示某种资源数

整型信号量与普通整型变量的区别：对信号量只能执行 初始化、P、V 三种操作

整型信号量存在的问题：不满足让权等待原则

记录型信号量

S.value 表示某种资源数，S.L 指向等待该资源的队列

P 操作中，一定是先 S.value--，之后可能需要执行 block 原语

V 操作中，一定是先 S.value++，之后可能需要执行 wakeup 原语

注意：要能够自己推断在什么条件下需要执行 block 或 wakeup

可以用记录型信号量实现系统资源的“申请”和“释放”

可以用记录型信号量实现进程互斥、进程同步

大题、小题超
高频出题点

注：若考试中出现 P(S)、V(S) 的操作，除非特别说明，否则默认 S 为记录型信号量。

知识回顾与重要考点

除了互斥、同步问题外，还会考察有多个资源的问题，有多少资源就把信号量初值设为多少。申请资源时进行P操作，释放资源时进行V操作即可

信号量机制

实现进程互斥

分析问题，确定临界区

设置互斥信号量，初值为1

互斥问题，信号量初值为1

临界区之前对信号量执行 P 操作

临界区之后对信号量执行 V 操作

实现进程同步

分析问题，找出哪里需要实现“一前一后”的同步关系

设置同步信号量，初始值为0

同步问题，信号量初值为0

在“前操作”之后执行 V 操作

在“后操作”之前执行 P 操作

实现进程的前驱关系

分析问题，画出前驱图，把每一对前驱关系都看成一个同步问题

为每一对前驱关系设置同步信号量，初值为0

前驱关系问题，本质上就是多级同步问题

在每个“前操作”之后执行 V 操作

在每个“后操作”之前执行 P 操作

知识回顾与重要考点

管程

为什么要引入管程



解决信号量机制编程麻烦、易出错的问题

组成



共享数据结构

对数据结构初始化的语句

一组用来访问数据结构的过程（函数）

基本特征



各外部进程/线程只能通过管程提供的特定“入口”才能访问共享数据



每次仅允许一个进程在管程内执行某个内部过程

补充



各进程必须互斥访问管程的特性是由编译器实现的

可在管程中设置条件变量及等待/唤醒操作以解决同步问题

本节内容

生产者消费者
者问题

本节内容

读者-写者
问题

本节内容

哲学家进餐
问题

知识回顾与重要考点

死锁的概念

什么是死锁

各进程互相等待对方手里的资源，导致各进程都阻塞，无法向前推进

死锁、饥饿、死循环的区别

死锁：至少是两个进程一起死锁，死锁进程处于阻塞态

饥饿：可以只有一个进程饥饿，饥饿进程可能阻塞也可能就绪

死循环：可能只有一个进程发生死循环，死循环的进程可上处理机

死锁和饥饿是操作系统要解决的问题，死循环是应用程序员要解决的

死锁产生的必要条件

互斥条件

对必须互斥使用的资源的争抢才会导致死锁

不剥夺条件

进程保持的资源只能主动释放，不可强行剥夺

请求和保持条件

保持着某些资源不放的同时，请求别的资源

循环等待条件

存在一种进程资源的循环等待链

循环等待未必死锁，死锁一定有循环等待

什么时候会发生死锁

对不可剥夺资源的不合理分配，可能导致死锁

死锁的处理策略

预防死锁

破坏死锁产生的四个必要条件

避免死锁

避免系统进入不安全状态（银行家算法）

死锁的检测和解除

允许死锁发生，系统负责检测出死锁并解除

知识回顾与重要考点

预防死锁

破坏互斥条件

将临界资源改造为可共享使用的资源（如SPOOLing技术）

缺点：可行性不高，很多时候无法破坏互斥条件

破坏不剥夺条件

方案一，申请的资源得不到满足时，立即释放拥有的所有资源

方案二，申请的资源被其他进程占用时，由操作系统协助剥夺（考虑优先级）

缺点：实现复杂；剥夺资源可能导致部分工作失效；反复申请和释放导致系统开销大；可能导致饥饿

破坏请求和保持条件

运行前分配好所有需要的资源，之后一直保持

缺点：资源利用率低；可能导致饥饿

破坏循环等待条件

给资源编号，必须按编号从小到大的顺序申请资源

缺点：不方便增加新设备；会导致资源浪费；用户编程麻烦

知识回顾与重要考点

数据结构:

长度为 m 的一维数组 **Available** 表示还有多少可用资源

$n \times m$ 矩阵 **Max** 表示各进程对资源的最大需求数

$n \times m$ 矩阵 **Allocation** 表示已经给各进程分配了多少资源

$\text{Max} - \text{Allocation} = \text{Need}$ 矩阵表示各进程最多还需要多少资源

用长度为 m 的一位数组 **Request** 表示进程此次申请的各种资源数

银行家算法步骤:

- ①检查此次申请是否超过了之前声明的最大需求数
- ②检查此时系统剩余的可用资源是否还能满足这次请求
- ③试探着分配, 更改各数据结构
- ④用安全性算法检查此次分配是否会导致系统进入不安全状态

安全性算法步骤:

检查当前的剩余可用资源是否能满足某个进程的最大需求, 如果可以, 就把该进程加入安全序列, 并把该进程持有的资源全部回收。

不断重复上述过程, 看最终是否能让所有进程都加入安全序列。

系统处于不安全状态未必死锁, 但死锁时一定处于不安全状态。系统处于安全状态一定不会死锁。

知识回顾与重要考点

死锁的检测和解除

如何检测

数据结构: 资源分配图

两种结点

进程结点

资源结点

两种边

进程结点——>资源结点 (请求边)

资源节点——>进程结点 (分配边)

死锁检测算法

依次消除与不阻塞进程相连的边, 直到无边可消

注: 所谓不阻塞进程是指其申请的资源数还足够的进程

死锁定理: 若资源分配图是不可完全简化的, 说明发生了死锁

如何解除

资源剥夺法

撤销进程法 (终止进程法)

进程回退法

题中一般会给出资源分配图。不过也要小心与数据结构结合考察。

知识回顾与重要考点

内存的基础知识



注意别名: 静态重定位、动态重定位

知识回顾与重要考点

内存管理的概念

内存空间的分配与回收

内存空间的扩充（实现虚拟性）

地址转换

操作系统负责实现逻辑地址到物理地址的转换

三种方式

绝对装入：编译器负责地址转换（单道程序阶段，无操作系统）

可重定位装入：装入程序负责地址转换（早期多道批处理阶段）

动态运行时装入：运行时才进行地址转换（现代操作系统）

存储保护

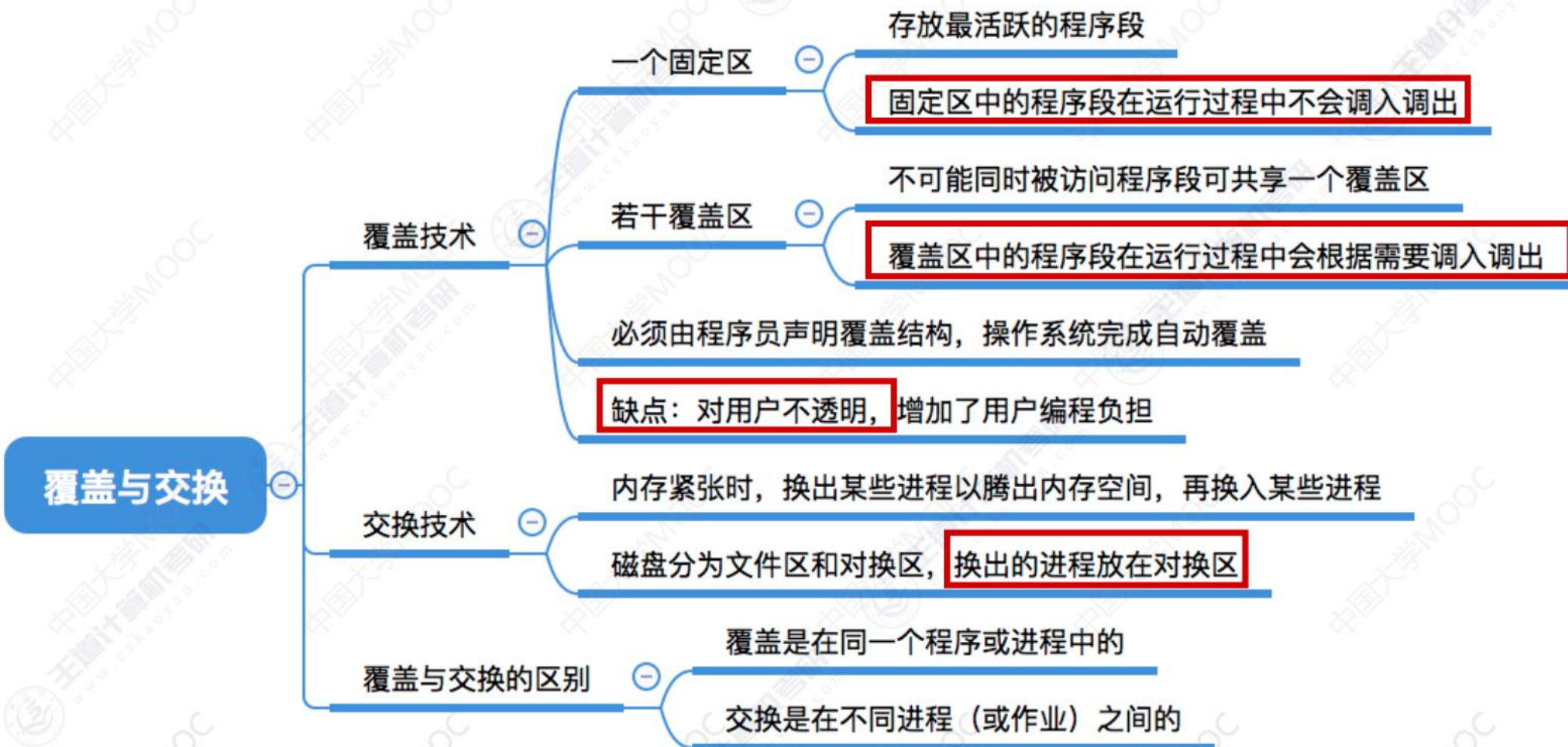
保证各进程在自己的内存空间内运行，不会越界访问

两种方式

设置上下限寄存器

利用重定位寄存器、界地址寄存器进行判断

知识回顾与重要考点



知识回顾与重要考点

连续分配管理

为用户进程分配的必须是一个连续的内存空间

另外，需要对用于管理空闲分区的数据结构有印象——空闲分区表、空闲分区链

单一连续分配

只支持单道程序，内存分为系统区和用户区，用户程序放在用户区

无外部碎片，有内部碎片

固定分区分配

支持多道程序，内存用户空间分为若干个固定大小的分区，每个分区只能装一道作业

无外部碎片，有内部碎片

两种分区方式

分区大小相等

分区大小不等

动态分区分配

支持多道程序，在进程装入内存时，根据进程的大小动态地建立分区

无内部碎片，有外部碎片

外部碎片可用“紧凑”技术来解决

回收内存分区时，可能遇到四种情况

回收区之后有相邻的空闲分区

回收区之前有相邻的空闲分区

回收区前、后都有相邻的空闲分区

回收区前、后都没有相邻的空闲分区

总之，相邻的空闲分区要合并

知识回顾与重要考点

算法	算法思想	分区排列顺序	优点	缺点
首次适应	从头到尾找适合的分区	空闲分区以地址递增次序排列	综合看性能最好。算法开销小，回收分区后一般不需要对空闲分区队列重新排序	
最佳适应	优先使用更小的分区，以保留更多大分区	空闲分区以容量递增次序排列	会有更多的大分区被保留下来，更能满足大进程需求	会产生很多太小的、难以利用的碎片；算法开销大，回收分区后可能需要对空闲分区队列重新排序
最坏适应	优先使用更大的分区，以防止产生太小的不可用的碎片	空闲分区以容量递减次序排列	可以减少难以利用的小碎片	大分区容易被用完，不利于大进程；算法开销大（原因同上）
邻近适应	由首次适应演变而来，每次从上次查找结束位置开始查找	空闲分区以地址递增次序排列（可排列成循环链表）	不用每次都从低地址的小分区开始检索。算法开销小（原因同首次适应算法）	会使高地址的大分区也被用完

知识回顾与重要考点

基本分页存储管理的思想：把进程分页，各个页面可离散地放到各个的内存块中

基本分页存储管理的基本概念

易混概念

“页框、页帧、内存块、物理块、物理页” VS “页、页面”

“页框号、页帧号、内存块号、物理块号、物理页号” VS “页号、页面号”

页表

页表记录了页面和实际存放的内存块之间的映射关系

一个进程对应一张页表，进程的每一页对应一个页表项，每个页表项由“页号”和“块号”组成

每个页表项的大小是相同的，页号是“隐含”的

i 号页表项存放地址 = 页表始址 + i * 页表项大小

内存块的数量 → 页表项大小

逻辑地址结构——可拆分为【页号P，页内偏移量W】

页号 = 逻辑地址 / 页面大小；页内偏移量 = 逻辑地址 % 页面大小

如果页面大小刚好是2的整数次幂呢？

如何实现地址转换

1. 计算出逻辑地址对应的【页号，页内偏移量】

2. 找到对应页面在内存中的存放位置（查页表）

3. 物理地址 = 页面始址 + 页内偏移量

知识回顾与重要考点

基本地址变换机构

页表寄存器的作用

存放页表起始地址

存放页表长度

地址变换过程

1. 根据逻辑地址算出页号、页内偏移量

2. 页号的合法性检查（与页表长度对比）

第一次访问内存：查页表

3. 若页号合法，再根据页表起始地址、页号找到对应页表项

4. 根据页表项中记录的内存块号、页内偏移量 得到最终的物理地址

5. 访问物理内存对应的内存单元

第二次访问内存：
访问目标内存单元

其他小细节

页内偏移量位数与页面大小之间的关系（要能用其中一个条件推出另一个条件）

页式管理中地址是一维的

实际应用中，通常使一个页框恰好能放入整数个页表项

为了方便找到页表项，页表一般是放在连续的内存块中的

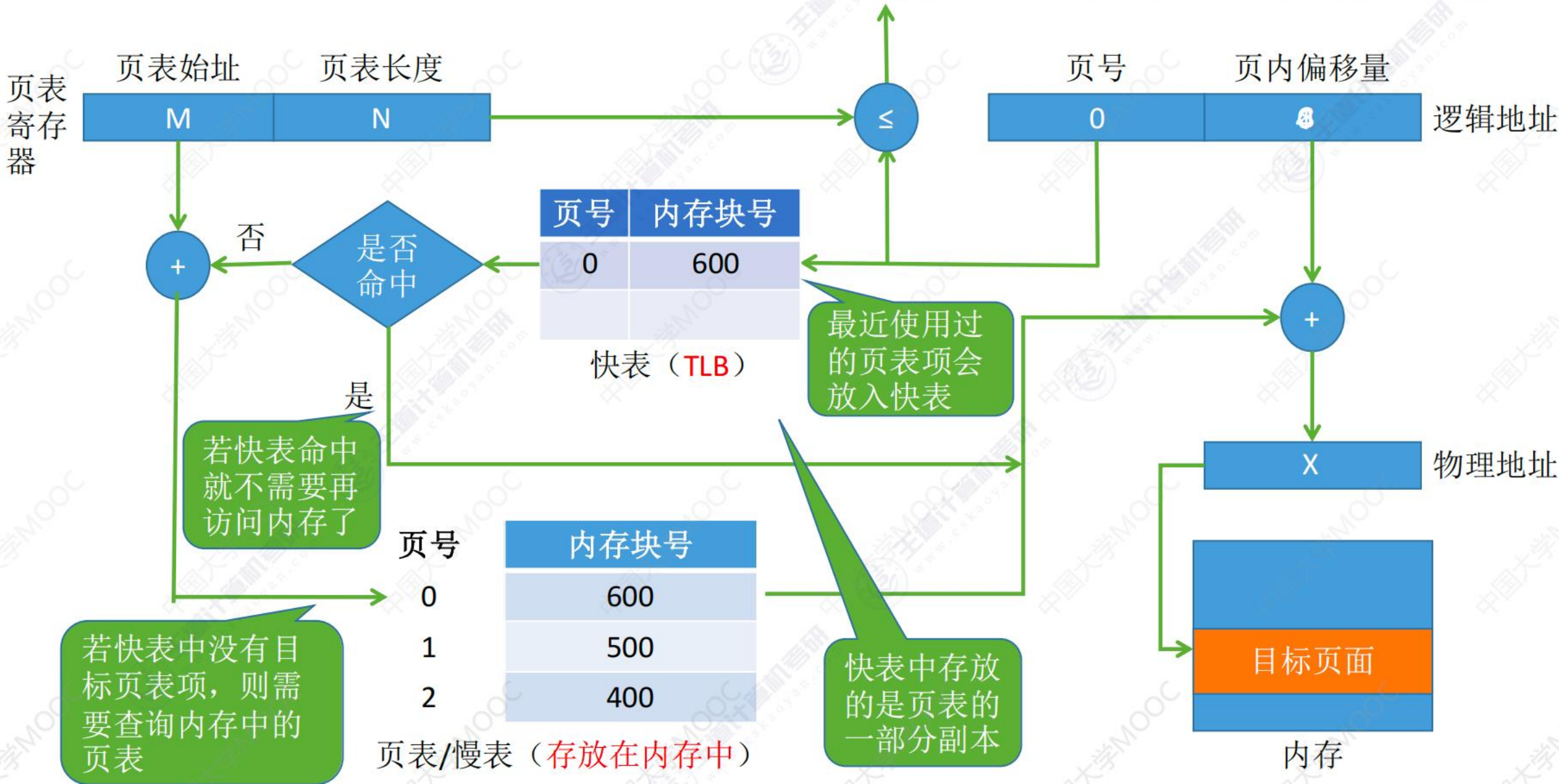
知识回顾与重要考点

	地址变换过程	访问一个逻辑地址的访存次数
基本地址变换机构	①算页号、页内偏移量 ②检查页号合法性 ③查页表，找到页面存放的内存块号 ④根据内存块号与页内偏移量得到物理地址 ⑤访问目标内存单元	两次访存
具有快表的地址变换机构	①算页号、页内偏移量 ②检查页号合法性 ③查快表。若命中，即可知道页面存放的内存块号，可直接进行⑤；若未命中则进行④ ④查页表，找到页面存放的内存块号，并且将页表项复制到快表中 ⑤根据内存块号与页内偏移量得到物理地址 ⑥访问目标内存单元	快表命中，只需一次访存 快表未命中，需要两次访存

TLB 和普通 Cache 的区别——TLB 中只有页表项的副本，而普通 Cache 中可能会有其他各种数据的副本

假设：访问TLB只需1 us
访问内存需要100 us

假设某进程执行过程中要依次访问
(0, 0)、(0, 4)、(0, 8) 这几个逻辑地址



知识回顾与重要考点

两级页表

单级页表存在的问题

所有页表项必须连续存放，页表过大时需要很大的连续空间

在一段时间内并非所有页面都用得到，因此没必要让整个页表常驻内存

将长长的页表再分页

两级页表

⚠ 逻辑地址结构：（一级页号，二级页号，页内偏移量）

⚠ 注意几个术语：页目录表/外层页表/顶级页表

要能根据逻辑地址位数、页面大小、页表项大小 确定多级页表的逻辑地址结构

如何实现地址变换

①按照地址结构将逻辑地址拆分成三部分

②从PCB 中读出页目录表始址，根据一级页号查页目录表，找到下一级页表在内存中的存放位置

③根据二级页号查表，找到最终想访问的内存块号

④结合页内偏移量得到物理地址

几个细节

⚠ 多级页表中，各级页表的大小不能超过一个页面。若两级页表不够，可以分更多级

⚠ 多级页表的访存次数（假设没有快表机构）—— N 级页表访问一个逻辑地址需要 $N+1$ 次访存

知识回顾与重要考点

将地址空间按照程序自身的逻辑关系划分为若干个段，每段从0开始编址

分段

每个段在内存中占据连续空间，但各段之间可以不相邻

逻辑地址结构：（段号，段内地址）

段表

记录逻辑段到实际存储地址的映射关系

每个段对应一个段表项。各段表项长度相同，由段号（隐含）、**段长**、基址组成

基本分段存储管理

地址变换

1. 由逻辑地址得到段号、段内地址
2. 段号与段表寄存器中的段长度比较，检查是否越界
3. 由段表始址、段号找到对应段表项
4. **根据段表中记录的段长，检查段内地址是否越界**
5. 由段表中的“基址+段内地址”得到最终的物理地址
6. 访问目标单元

分段 VS 分页

分页对用户不可见，分段对用户可见

分页的地址空间是一维的，分段的地址空间是二维的

分段更容易实现信息的共享和保护（纯代码/可重入代码可以共享）

分页（单级页表）、分段访问一个逻辑地址都需要两次访存，分段存储中也可以引入快表机构

知识回顾与重要考点

将地址空间按照程序自身的逻辑关系划分为若干个段，再将各段分为大小相等的页面

分段+分页

将内存空间分为与页面大小相等的一个个内存块，系统以块为单位为进程分配内存

逻辑地址结构：（段号，页号，页内偏移量）

段表、页表

每个段对应一个段表项。各段表项长度相同，由段号（隐含）、**页表长度、页表存放地址**组成

每个页对应一个页表项。各页表项长度相同，由页号（隐含）、页面存放的内存块号组成

段页式管理

1. 由逻辑地址得到段号、页号、页内偏移量
2. 段号与段表寄存器中的段长度比较，检查是否越界
3. 由段表始址、段号找到对应段表项
4. **根据段表中记录的页表长度，检查页号是否越界**
5. 由段表中的页表地址、页号得到查询页表，找到相应页表项
6. 由页面存放的内存块号、页内偏移量得到最终的物理地址
7. 访问目标单元

地址变换

第一次——查段表、第二次——查页表、第三次——访问目标单元

访问一个逻辑地址所需访存次数

可引入快表机构，以段号和页号为关键字查询快表，即可直接找到最终的目标页面存放位置。引入快表后仅需一次访存

知识回顾与重要考点

虚拟内存的基本概念

传统存储管理方式的特征、缺点

一次性：作业数据必须一次全部调入内存

驻留性：作业数据在整个运行期间都会常驻内存

局部性原理

时间局部性：现在访问的指令、数据在不久后很可能会被再次访问

空间局部性：现在访问的内存单元周围的内存空间，很可能在不久后会被访问

高速缓存技术：使用频繁的数据放到更高速的存储器中

虚拟内存的定义和特征

程序不需全部装入即可运行，运行时根据需要动态调入数据，若内存不够，还需换出一些数据

特征

多次性：无需在作业运行时一次性全部装入内存，而是允许被分成多次调入内存。

对换性：无需在作业运行时一直常驻内存，而是允许在作业运行过程中，将作业换入、换出。

虚拟性：从逻辑上扩充了内存的容量，使用户看到的内存容量，远大于实际的容量。

如何实现虚拟内存技术

访问的信息不在内存时，由操作系统负责将所需信息从外存调入内存（请求调页功能）

内存空间不够时，将内存中暂时用不到的信息换出到外存（页面置换功能）

虚拟内存的实现

请求分页存储管理

请求分段存储管理

请求段页式存储管理

知识回顾与重要考点

请求分页管理方式

页表机制

在基本分页的基础上增加了几个表项

状态位：表示页面是否已在内存中

访问字段：记录最近被访问过几次，或记录上次访问的时间，供置换算法选择换出页面时参考

修改位：表示页面调入内存后是否被修改过，只有修改过的页面才需在置换时写回外存

外存地址：页面在外存中存放的位置

缺页中断机构

找到页表项后检查页面是否已在内存，若没在内存，产生缺页中断

缺页中断处理中，需要将目标页面调入内存，必要时还要换出页面

缺页中断属于内中断，属于内中断中的“故障”，即可能被系统修复的异常

一条指令在执行过程中可能产生多次缺页中断

地址变换机构（重点关注与基本分页不同的地方）

找到页表项是需要检查页面是否在内存中

若页面不再内存中，需要请求调页

若内存空间不够，还需换出页面

页面调入内存后，需要修改相应页表项

知识回顾与重要考点

	算法规则	优缺点
OPT	优先淘汰最长时间内不会被访问的页面	缺页率最小，性能最好；但无法实现
FIFO	优先淘汰最先进入内存的页面	实现简单；但性能很差，可能出现Belady异常
LRU	优先淘汰最近最久没访问的页面	性能很好；但需要硬件支持，算法开销大
CLOCK (NRU)	循环扫描各页面 第一轮淘汰访问位=0的，并将扫描过的页面访问位改为1。若第一轮没选中，则进行第二轮扫描。	实现简单，算法开销小；但未考虑页面是否被修改过。
改进型CLOCK (改进型NRU)	若用（访问位, 修改位）的形式表述，则 第一轮：淘汰（0, 0） 第二轮：淘汰（0, 1），并将扫描过的页面访问位都置为0 第三轮：淘汰（0, 0） 第四轮：淘汰（0, 1）	算法开销较小，性能也不错

知识回顾与重要考点

驻留集

指请求分页存储管理中给进程分配的内存块的集合

固定分配 VS 可变分配：区别在于进程运行期间驻留集大小是否可变

局部置换 VS 全局置换：区别在于发生缺页时是否只能从进程自己的页面中选择一个换出

页面分配、置换策略

固定分配局部置换：进程运行前就分配一定数量物理块，缺页时只能换出进程自己的某一页

可变分配全局置换：只要缺页就分配新物理块，可能来自空闲物理块，也可能需换出别的进程页面

可变分配局部置换：频繁缺页的进程，多分配一些物理块；缺页率很低的进程，回收一些物理块。直到缺页率合适

页面分配策略

何时调入页面

预调页策略：一般用于进程运行前

请求调页策略：进程运行时，发现缺页再调页

从何处调页

对换区——采用连续存储方式，速度更快；文件区——采用离散存储方式，速度更慢。

对换区足够大：运行将数据从文件区复制到对换区，之后所有的页面调入、调出都是在内存与对换区之间进行

对换区不够大：不会修改的数据每次都从文件区调入；会修改的数据调出到对换区，需要时再从对换区调入

UNIX方式：第一次使用的页面都从文件区调入；调出的页面都写回对换区，再次使用时从对换区调入

抖动（颠簸）现象

页面频繁换入换出的现象。主要原因是分配给进程的物理块不够

工作集

在某段时间间隔里，进程实际访问页面的集合。驻留集大小一般不能小于工作集大小

知识点回顾与重要考点

初识文件管理

文件的定义：一组有意义的信息的集合

文件的属性：文件名、标识符、类型、位置、大小、保护信息...

文件内部应该如何被组织起来（文件的逻辑结构）

文件之间应该如何被组织起来（目录结构）

操作系统应向上提供哪些功能（create、delete、open、close、read、write 系统调用）

文件应如何存放在外存中（文件的物理结构）

操作系统如何管理外存中的空闲块（存储空间的管理）

操作系统需要提供的其他文件管理功能

文件共享

文件保护

知识点回顾与重要考点

文件的逻辑结构

无结构文件



由二进制流或字符流组成，无明显的逻辑结构

由记录组成，分为定长记录、可变长记录

有结构文件



逻辑结构



顺序文件

索引文件

索引顺序文件

知识点回顾与重要考点

有结构文件

顺序文件

默认各记录在物理上顺序存储

串结构：记录顺序与关键字无关

顺序结构：记录按关键字顺序排列

可变长记录的顺序文件在每次查询时只能从头依次查找

可变长记录的顺序文件无法实现随机存取，定长记录可以

定长记录、顺序结构的顺序文件可以快速检索（根据关键字快速找到记录）

最大缺点：不方便增加/删除记录

索引文件

建立一张索引表，每个记录对应一个表项。各记录不用保持顺序，方便增加/删除记录

索引表本身就是定长记录的顺序文件，一个索引表项就是一条定长记录，因此索引文件可支持随机存取

若索引表按关键字顺序排列，则可支持快速检索

解决了顺序文件不方便增/删记录的问题，同时让不定长记录的文件实现了随机存取。但索引表可能占用很多空间

索引顺序文件

将记录分组，每组对应一个索引表项

检索记录时先顺序查索引表，找到分组，再顺序查找分组

当记录过多时，可建立多级索引表

要会计算平均查找次数

知识点回顾与重要考点

文件目录

文件目录的实现

一个文件对应一个FCB，一个FCB就是一个目录项，多个FCB组成文件目录

对目录的操作：搜索、创建文件、删除文件、显示文件、修改文件

单级目录结构

一个系统只有一张目录表，不允许文件重名

两级目录结构

不同用户的文件可以重名，但不能对文件进行分类

目录结构

多级（树形）目录结构

不同目录下的文件可以重名，可以对文件进行分类，不方便文件共享

系统根据“文件路径”找到目标文件

从根目录出发的路径是“绝对路径”（“/照片/2015-08/自拍.jpg”）

从“当前目录”出发的路径是“相对路径”（“照片/2015-08/自拍.jpg”）

无环图目录结构

在树形目录结构的基础上，增加一些指向同一节点的有向边，使整个目录成为一个有向无环图

为共享结点设置一个共享计数器，计数器为0时才真正删除该结点

索引结点

除了文件名之外的所有信息都放到索引结点中，每个文件对应一个索引结点

目录项中只包含文件名、索引结点指针，因此每个目录项的长度大幅减小

由于目录项长度减小，因此每个磁盘块可以存放更多个目录项，因此检索文件时磁盘I/O的次数就少了很多

要理解为什么根据“相对路径”检索文件可以减少磁盘I/O次数

理解

知识点回顾与重要考点

		How?	目录项内容	优点	缺点
顺序分配		为文件分配的必须是连续的磁盘块	起始块号、文件长度	顺序存取速度快, 支持随机访问	会产生碎片, 不利于文件拓展
链接分配	隐式链接	除文件的最后一个盘块之外, 每个盘块中都存有指向下一个盘块的指针	起始块号、结束块号	可解决碎片问题, 外存利用率高, 文件拓展实现方便	只能顺序访问, 不能随机访问。
	显式链接	建立一张文件分配表(FAT), 显式记录盘块的先后关系 (开机后FAT常驻内存)	起始块号	除了拥有隐式链接的优点之外, 还可通过查询内存中的FAT实现随机访问	FAT需要占用一定的存储空间
索引分配		为文件数据块建立索引表。若文件太大, 可采用链接方案、多层索引、混合索引	链接方案记录的是第一个索引块的块号, 多层/混合索引记录的是顶级索引块的块号	支持随机访问, 易于实现文件的拓展	索引表需占用一定的存储空间。访问数据块前需要先读入索引块。若采用链接方案, 查找索引块时可能需要很多次读磁盘操作。

知识点回顾与重要考点

文件存储空间管理

文件卷（逻辑卷），目录区、文件区的概念

存储空间的划分与初始化

目录区包含文件目录、空闲表、位示图、超级块等用于文件管理的数据

空闲表法

空闲表中记录每个连续空闲区的起始盘块号、盘块数

分配时可采用首次适应、最佳适应等策略；回收时注意表项的合并问题

空闲链表法

空闲盘块链

以盘块为单位组成一条空闲链

分配时从链头依次取出空闲块，回收时将空闲块查到链尾

空闲盘区链

以盘区为单位组成一条空闲链

分配时可采用首次适应、最佳适应等策略；回收时注意相邻空闲盘区合并的问题

位示图法

一个二进制位对应一个盘块。（字号，位号）或（行号，列号）与盘块号一一对应

重要考点：要能够自己推出盘块号 $\rightarrow$ （字号，位号）之间的相互转换公式

需要注意的题目条件

二进制位 0/1 到底哪个代表空闲，哪个代表不空闲

字号、位号、盘块号到底是从0开始还是从1开始

成组链接法

UNIX 采用的策略，适合大型文件系统。理解即可，不方便用文字描述的知识点也很难作为考题

知识点回顾与重要考点

文件共享

硬链接

各个用户的目录项指向同一个索引结点

索引结点中需要有链接计数 count

某用户想删除文件时，只是删除该用户的目录项，且count--

只有 $\text{count} == 0$ 时才能真正删除文件数据和索引结点，否则会导致指针悬空

软链接（符号链接）

在一个 Link 型的文件中记录共享文件的存放路径（Windows 快捷方式）

操作系统根据路径一层层查找目录，最终找到共享文件

即使软链接指向的共享文件已被删除，Link 型文件依然存在，只是通过 Link 型文件中的路径去查找共享文件会失败（找不到对应目录项）

由于用软链接的方式访问共享文件时要查询多级目录，会有多次磁盘I/O，因此用软链接访问共享文件的速度要比硬链接更慢

知识点回顾与重要考点

磁盘的结构

磁盘、磁道、扇区的概念

磁盘由表面涂有磁性物质的圆形盘片组成

每个盘片被划分为一个个磁道，每个磁道又划分为一个个扇区

如何在磁盘中读/写数据

磁头移动到目标位置，盘片旋转，对应扇区划过磁道才能完成读/写

盘面、柱面的概念

磁盘有多个盘片“摞”起来，每个盘片有两个盘面

所有盘面中相对位置相同的磁道组成柱面

磁盘的物理地址

(柱面号，盘面号，扇区号)

磁盘的分类

根据磁头是否可移动

固定头磁盘 (每个磁道有一个磁头)

移动头磁盘 (每个盘面只有一个磁头)

根据盘片是否可更换

固定盘磁盘

可换盘磁盘

知识点回顾与重要考点

磁盘调度算法

一次磁盘读/写操作需要的时间

寻找时间(寻道时间): 启动磁臂 **移动磁头所花的时间**

磁盘调度算法影响的指标

延迟时间: 将目标扇区转到磁头下面所花的时间

传输时间: 读/写 数据花费的时间

先来先服务 (FCFS)

按访问请求到达的先后顺序进行处理

最短寻找时间优先 (SSTF)

每次都优先响应距离磁头最近的磁道访问请求

贪心算法的思想, 能保证眼前最优, 但无法保证总的寻道时间最短

缺点: 可能导致饥饿

磁盘调度算法

扫描算法 (电梯算法、SCAN)

只有磁头移动到最边缘的磁道时才可以改变磁头移动方向

缺点: 对各个位置磁道的响应频率不平均

循环扫描算法 (C-SCAN)

只有磁头朝某个方向移动时才会响应请求, 移动到边缘后立即让磁头返回起点, 返回途中不响应任何请求

LOOK 算法

SCAN 算法的改进, 只要在磁头移动方向上不再有请求, 就立即改变磁头方向

低频考点

C-LOOK 算法

C-SCAN 算法的改进, 只要在磁头移动方向上不再有请求, 就立即让磁头返回

若题目中无特别说明, 则SCAN 就是 LOOK, C-SCAN 就是C-LOOK

知识点回顾与重要考点

I/O设备的基本概念与分类

什么是I/O设备

将数据 Input/Output (输入/输出) 计算机的外部设备

按使用特性分类

人机交互类外部设备

存储设备

网络通信设备

按传输速率分类

低速设备

中速设备

高速设备

按信息交换的单位分类

块设备 (传输快, 可寻址)

字符设备 (传输慢, 不可寻址, 常采用中断驱动方式)

一种I/O控制方式

知识点回顾与重要考点

用于实现
对I/O设备的
控制

I/O控制器

I/O设备由机
械部件和电
子部件组成

主要功能

接受和识别CPU发出的命令（要有控制寄存器）

向CPU报告设备的状态（要有状态寄存器）

数据交换（要有数据寄存器，暂存输入/输出的数据）

地址识别（由I/O逻辑实现）

组成

CPU与控制器之间的接口（实现控制器与CPU之间的通信）

I/O逻辑（负责识别CPU发出的命令，并向设备发出命令）

控制器与设备之间的接口（实现控制器与设备之间的通信）

两种寄存器编址方式

内存映射I/O

控制器中的寄存器与内存统一编制

可以采用对内存进行操作的指令来对控制器进行操作

寄存器独立编制

控制器中的寄存器独立编制

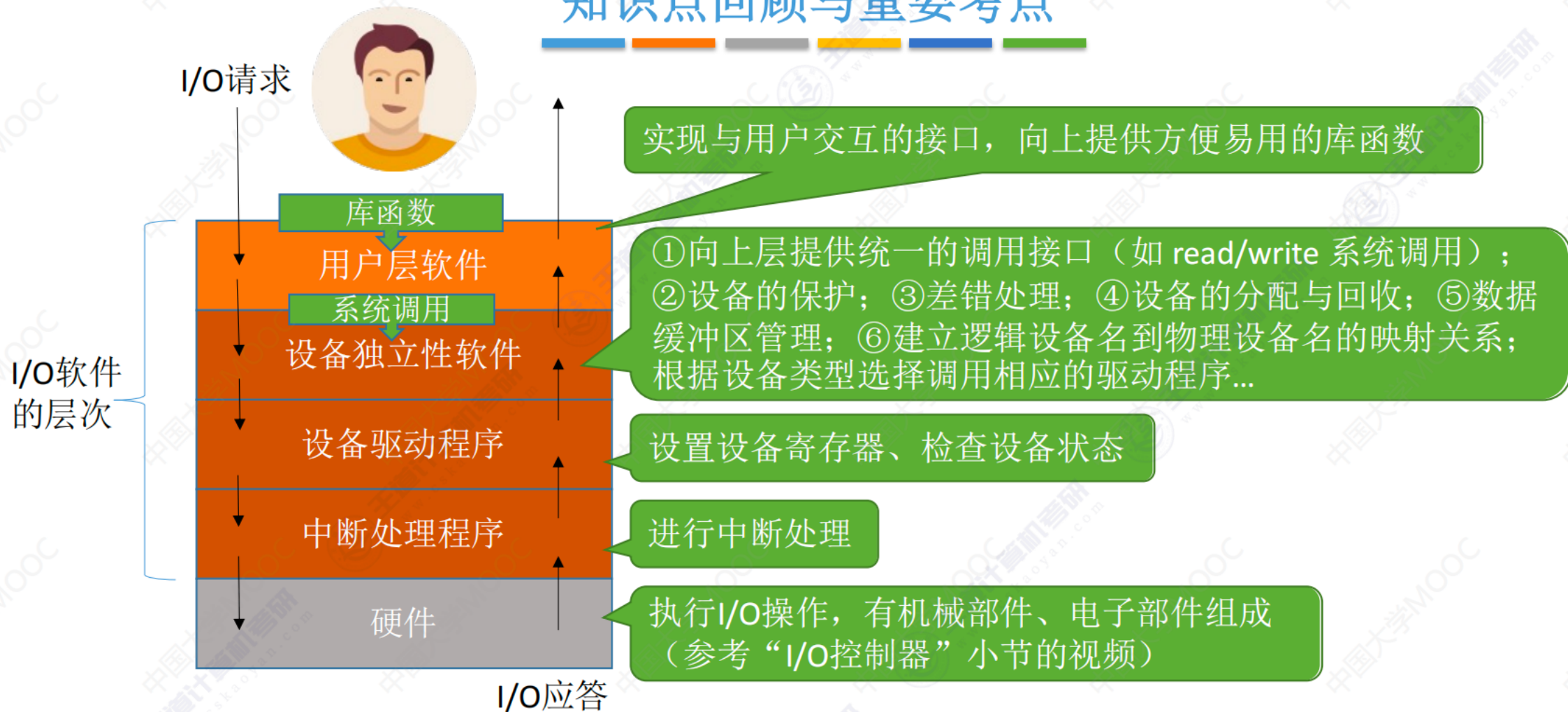
需要设置专门的指令来操作控制器

知识点回顾与重要考点

	完成一次读/写的过程	CPU干 预频率	每次I/O的数 据传输单位	数据流向	优缺点
程序直接控 制方式	CPU发出I/O命令后需要不 断轮询	极高	字	设备→CPU→内存 内存→CPU→设备	每一个阶段的 优点都是解决 了上一阶段的 最大缺点。 总体来说，整 个发展过程就 是要尽量减少 CPU对I/O过程 的干预，把CPU 从繁杂的I/O控 制事务中解脱 出来，以便更 多地去完成数 据处理任务。
中断驱动方 式	CPU发出I/O命令后可以 做其他事，本次I/O完成后 设备控制器发出中断信号	高	字	设备→CPU→内存 内存→CPU→设备	
DMA方式	CPU发出I/O命令后可以 做其他事，本次I/O完成后 DMA控制器发出中断信号	中	块	设备→内存 内存→设备	
通道控制方 式	CPU发出I/O命令后可以 做其他事。通道会执行通道 程序以完成I/O，完成后通 道向CPU发出中断信号	低	一组块	设备→内存 内存→设备	

难点理解：
通道=弱鸡版CPU

知识点回顾与重要考点



理解并记住I/O软件各个层次之间的顺序，要能够推理判断某个处理应该是在哪个层次完成的（最常考的是设备独立性软件、设备驱动程序这两层。只需理解一个特点即可：直接涉及到硬件具体细节、且与中断无关的操作肯定是在设备驱动程序层完成的；没有涉及硬件的、对各种设备都需要进行的管理工作都是在设备独立性软件层完成的）

知识总览

假脱机技术/ SPOOLing技术

脱机技术

外围控制机+更高速的设备——磁带

作用：缓解设备与CPU的速度矛盾，实现预输入、缓输出

假脱机技术

又叫SPOOLing技术，用软件的方式模拟脱机技术

输入井和输出井——模拟脱机输入/输出时的磁带

输入进程和输出进程——模拟脱机输入/输出时的外围控制机

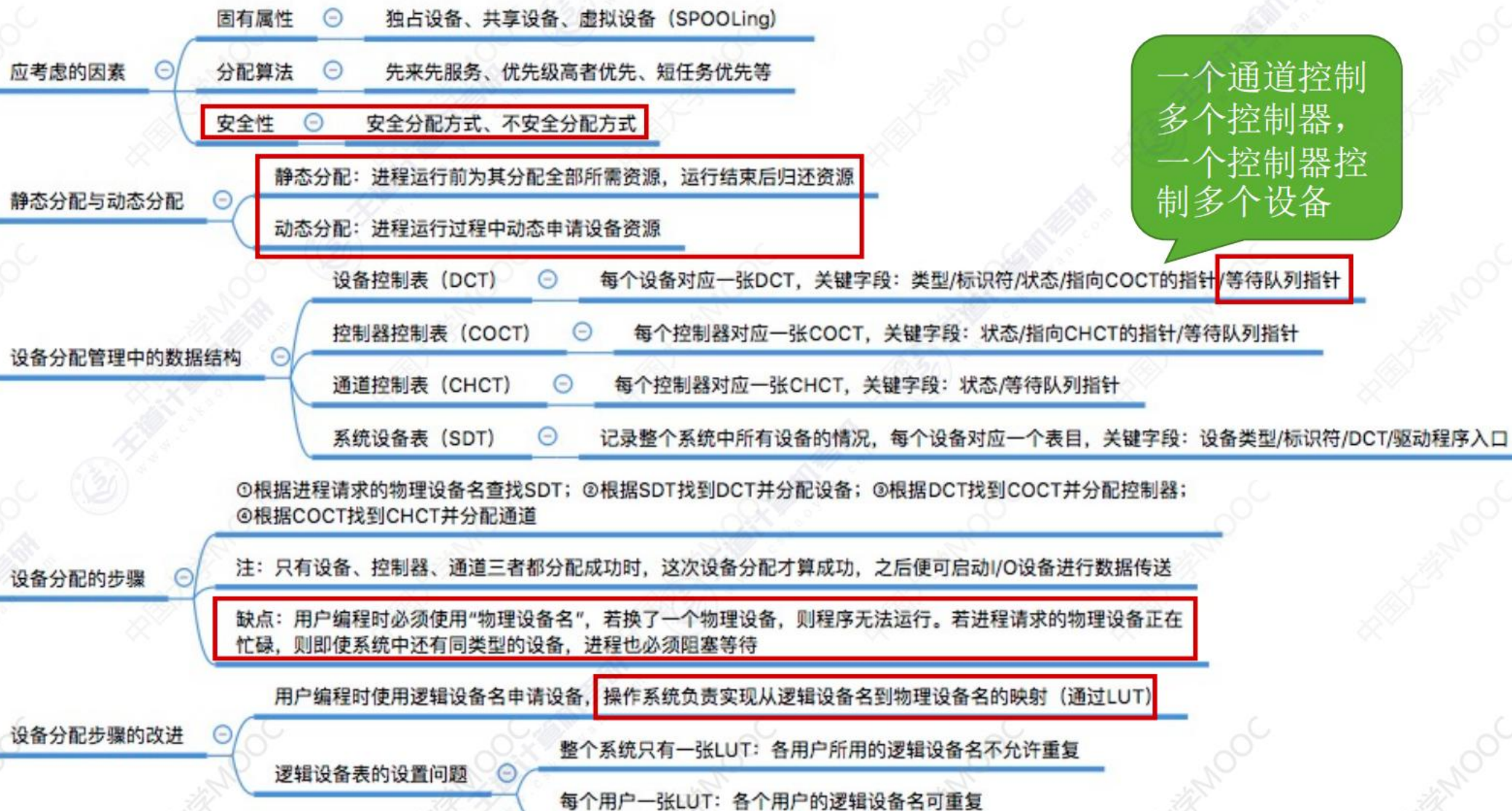
输入缓冲区和输出缓冲区——内存中的缓冲区，输入、输出时的“中转站”

共享打印机

用SPOOLing技术将独占式的打印机“虚拟”成共享打印机

知识总览

设备的分配与回收



一个通道控制多个控制器，一个控制器控制多个设备

知识总览

缓冲区管理

什么是缓冲区？有什么作用？

单缓冲

双缓冲

循环缓冲

缓冲池

什么是缓冲区？有什么作用？

缓冲区是一个存储区域，可以由专门的硬件寄存器组成，也可利用内存作为缓冲区。

使用**硬件作为缓冲区**的**成本较高**，**容量也较小**，一般仅用在对速度要求非常高的场合（如存储器管理中所用的联想寄存器，由于对页表的访问频率极高，因此使用速度很快的联想寄存器来存放页表项的副本）

一般情况下，更多的是利用**内存作为缓冲区**，“设备独立性软件”的缓冲区管理就是要组织管理好这些缓冲区

本节介绍的是“内存作为缓冲区”

缓冲区的作用

缓和CPU与I/O设备之间速度不匹配的矛盾

减少对CPU的中断频率，放宽对CPU中断响应时间的限制

解决数据粒度不匹配的问题

提高CPU与I/O设备之间的并行性

知识总览

一般利用内存作为缓冲区

缓冲区的概念

缓解CPU与设备的速度矛盾、减少对CPU的中断频率、解决数据粒度不匹配的问题、提高CPU与I/O设备之间的并行性

设备—(T)—缓冲区—(M)—工作区—(C)—处理

单缓冲

处理一块数据平均耗时 $\text{Max}(C, T) + M$

分析问题的初始状态：工作区满，缓冲区空

双缓冲

处理一块数据平均耗时 $\text{Max}(T, C + M)$

分析问题的初始状态：工作区空，一个缓冲区满，另一个缓冲区空

循环缓冲

多个缓冲区链接成循环队列，in指针指向第一个空缓冲区，out指针指向第一个满缓冲区

缓冲池

三个队列：空缓冲队列、输入队列、输出队列

四种工作缓冲区

用于收容输入数据的工作缓冲区、用于提取输入数据的工作缓冲区

用于收容输出数据的工作缓冲区、用于提取输出数据的工作缓冲区

缓冲区管理